

魔方解法与理论

从第一性原理出发的综合视角

综合层先法、CFOP、Roux、ZZ、Petrus、盲拧与 Kociemba 方法
基于 Singmaster、Frey、Joyner、Fridrich 等人的工作

本文从群论这一第一性原理出发，系统梳理三阶魔方的数学结构与主流解法。首先建立魔方的置换群模型与不变量理论，继而以换位子与共轭为统一工具，剖析层先法、CFOP、Roux、ZZ、Petrus、盲拧系列与 Kociemba 两阶段算法的内在机制，最终归纳各方法在策略层面的共性与权衡。

2026 年 6 月 27 日

目录

1 引言：魔方与第一性原理	4
1.1 魔方的历史与本质	4
1.2 什么是“第一性原理”视角	4
1.3 本文的组织与读者指南	5
2 魔方的数学模型：群论基础	5
2.1 魔方作为一个置换群	5
2.2 状态空间与合法状态	6
2.3 基本操作与生成元	6
2.4 子群与不变量	6
2.5 奇偶性与约束的推论	7
2.6 God's Number 与下界	7
3 第一性原理：换位子与共轭	8
3.1 换位子的定义与直觉	8
3.2 共轭的作用	8
3.3 三循环换位子	9
3.4 从换位子构造算法	9
3.5 换位子方法的一般框架	9
4 层先法 (LBL)：从零开始的解法	10
4.1 思想：分层解决与“保留已完成”	10
4.2 第一层：底十字与底角	11
4.3 第二层：中层棱块	11
4.4 第三层：顶十字、顶面、顶角、顶棱	11
4.5 LBL 的算法表与记忆策略	12
4.6 评价：LBL 的优势与局限	12
5 CFOP 方法：速度魔方的基石	13
5.1 思想：LBL 的并行化与算法化	13
5.2 Cross：底十字的优化	13
5.3 F2L：First Two Layers 的成对插入	14
5.4 OLL：Orient Last Layer 的一步定向	14
5.5 PLL：Permute Last Layer 的一步归位	15

5.6	算法量与学习曲线	15
5.7	评价：CFOP 的优势与局限	16
6	Roux 方法：块构建的艺术	16
6.1	思想：块构建与桥式结构	16
6.2	First Block（第一块）	17
6.3	Second Block（第二块）	17
6.4	CMLL: Corners of Last Layer	17
6.5	LSE: Last Six Edges	17
6.6	评价：Roux 的优势与局限	18
7	ZZ 方法：无旋转的优雅	18
7.1	思想：边定向预处理	18
7.2	EOLine: 边定向 + 线	19
7.3	ZZ-F2L: 无旋转的前两层	19
7.4	LL: 最后一层	20
7.5	评价：ZZ 的优势与局限	20
8	Petrus 方法：最小破坏的策略	21
8.1	思想：逐步构建与保留定向	21
8.2	$2 \times 2 \times 2$ 块	21
8.3	扩展到 $2 \times 2 \times 3$	22
8.4	棱块定向 (EO)	22
8.5	完成 F2L 与 LL	22
8.6	评价：Petrus 的优势与局限	22
9	盲拧方法：序列化与缓冲区	22
9.1	思想：将求解转化为序列执行	23
9.2	Old Pochmann: 经典对换方法	23
9.3	M2 方法: M 层对换	24
9.4	3-Style: 换位子方法的高级应用	24
9.5	评价：盲拧方法的演进与权衡	24
10	计算机解法：Kociemba 两阶段算法	25
10.1	思想：状态空间剪枝与子群分解	25
10.2	Phase 1: 进入子群 G_1	25
10.3	Phase 2: 在 G_1 中求解	26

10.4 启发式与剪枝表的设计原理	26
10.5 两阶段算法的优化与变体	27
10.6 评价：Kociemba 算法的意义与局限	27
11 方法关联与内在机制	27
11.1 共同的数学骨架	27
11.2 分层 vs 块构建 vs 定向优先	28
11.3 算法库 vs 即兴构造	28
11.4 速度与记忆的权衡	29
11.5 第一性原理的统一视角	29
12 策略总结与学习路径	30
12.1 学习路径建议	30
12.2 不同目标的策略选择	31
12.3 进阶方向	32
12.4 结语：第一性原理的力量	32

1 引言：魔方与第一性原理

1.1 魔方的历史与本质

三阶魔方 (Rubik's Cube) 由匈牙利建筑学教授 Ernő Rubik 于 1974 年发明，最初作为帮助学生理解三维空间几何的教学工具。1980 年由 Ideal Toy 公司推向全球市场后，魔方迅速成为有史以来最畅销的智力玩具之一，销量超过四亿五千万件。然而，魔方的意义远不止于玩具：它是一个具有有限但极其庞大的状态空间的物理系统，其结构与操作天然地构成一个群，因而成为群论、组合数学与算法研究中极具价值的具象模型。

从本质上看，魔方是一个由 26 个可见小块 (8 个角块、12 个棱块、6 个中心块) 构成的立方体结构。每个面可以绕其中心轴作 90° 、 180° 或 270° 旋转，每次旋转都会重新排列部分小块的位置与朝向。中心块的位置相对于立方体骨架固定不变，它们定义了六个面的颜色与空间方位；而角块与棱块则在旋转中被置换并改变朝向。这一“位置置换 + 朝向变化”的双重结构，正是魔方群论模型的物理基础。

魔方的求解问题可以表述为：给定任意一个合法的打乱状态，找到一串基本旋转操作，使魔方回到所有面颜色一致的初始状态。这一问题的数学深度在于：状态空间极其庞大 (约 4.3×10^{19} 个合法状态)，但 God's Number 仅为 20——即任意状态都可在 20 步以内还原。这一巨大反差揭示了魔方结构的深层对称性，也构成了各类解法方法的理论起点。

1.2 什么是“第一性原理”视角

“第一性原理” (First Principles) 一词源自物理学与哲学，指不依赖经验类比与既有结论，而是从最基本、最不可约的事实与公理出发进行推理的思维方式。在魔方研究中，第一性原理视角意味着：不把任何具体解法 (如 CFOP 的某个公式、Roux 的某个块构建步骤) 视为天经地义，而是追问——魔方作为一个数学对象，其最根本的结构是什么？所有解法共同依赖的不可约原理是什么？

从这一视角出发，我们可以识别出魔方求解的三个第一性原理：

魔方求解的三个第一性原理

- 群结构原理**：魔方的所有合法状态构成一个群 $\mathcal{R}$ ，基本旋转是生成元，求解等价于在群中找到一条从当前状态到单位元的路径。
- 不变量原理**：并非所有看似可能的状态都是合法的；角块朝向之和、棱块朝向之和、角块置换与棱块置换的奇偶性这三个不变量约束了合法状态空间，使其恰好为全体状态的 $1/12$ 。
- 换位子原理**：任何只影响少数几个小块的“局部”操作都可以通过换位子 $[a, b] = aba^{-1}b^{-1}$ 及其共轭 gag^{-1} 系统地构造出来。这是从“暴力搜索”走向“算法构造”的关键桥梁。

这三个原理构成了本文的全部理论骨架。第二章建立群结构原理与不变量原理的严格形式化；第三章阐述换位子原理并展示其如何统一各类算法；第四至第十章则展示不同解法如何在这三个原理之上构建各自的策略；第十一章归纳各方法的关联与内在机制。

1.3 本文的组织与读者指南

本文面向对魔方已有初步实践、并希望深入理解其数学结构与解法原理的读者。全文共十二章，可分为四个部分：

第一部分（第 2–3 章）：理论基础。建立魔方的置换群模型，推导三大不变量，并系统介绍换位子与共轭这两个构造性工具。这部分是后续所有方法的数学公共基底，建议读者在阅读解法章节前充分掌握。

第二部分（第 4–8 章）：主流速拧方法。依次介绍层先法 (LBL)、CFOP、Roux、ZZ、Petrus 五种方法。每种方法均从其核心思想出发，详述各阶段步骤、关键算法与策略考量，并给出客观评价。

第三部分（第 9–10 章）：特殊场景方法。介绍盲拧系列方法 (Old Pochmann、M2、3-Style) 与计算机解法 (Kociemba 两阶段算法)。前者展示如何将求解转化为可记忆的序列执行，后者展示如何用群论子群结构实现近乎最优的求解。

第四部分（第 11–12 章）：综合与策略。第十一章从第一性原理视角归纳各方法的共同数学骨架与策略权衡；第十二章给出面向不同目标的学习路径建议。

阅读时，数学基础较弱的读者可先略过第二章的严格证明，重点理解换位子的构造思想；有竞速目标的读者可重点阅读第五至八章并结合第十二章选择适合自己的方法。

2 魔方的数学模型：群论基础

本章建立魔方的严格群论模型。我们将看到，魔方不仅“像”一个群，它本身就是有限群论中一个极为丰富的具体实例——其阶、子群结构、不变量与共轭类都为后续解法提供了直接的理论支撑。本章内容主要综合 Singmaster[1] 与 Frey–Singmaster[2] 的经典框架，并参考 Joyner[3] 的现代表述。

2.1 魔方作为一个置换群

定义 2.1 (魔方群). 三阶魔方的**魔方群** $\mathcal{R}$ 是所有可通过有限次基本面旋转复合得到的操作的集合，以复合为群运算。其生成元集为六个基本 90° 面旋转：

$$\mathcal{R} = \langle U, D, R, L, F, B \rangle,$$

其中 U, D, R, L, F, B 分别表示上、下、右、左、前、后面顺时针 90° 旋转。

每个面旋转本质上是对若干小块的置换。具体而言，一次 U 转会同时置换 4 个角块（位置循环）与 4 个棱块（位置循环），并改变这些小块的朝向（对 U 转而言，朝向不变；但对 F 转则会改变角块与棱块的朝向）。因此，魔方群的每一个元素可以分解为三个独立的分量：

魔方状态的三元组表示

任意魔方状态 $g \in \mathcal{R}$ 可表示为三元组

$$g = (\sigma_c, \sigma_e, \vec{v}),$$

其中：

- $\sigma_c \in \mathfrak{S}_8$ 是 8 个角块的位置置换；
- $\sigma_e \in \mathfrak{S}_{12}$ 是 12 个棱块的位置置换；
- $\vec{v} = (v_{c,1}, \dots, v_{c,8}, v_{e,1}, \dots, v_{e,12}) \in \mathbb{Z}_3^8 \times \mathbb{Z}_2^{12}$ 是角块朝向向量（每个角块有 3 种朝向）与棱块朝向向量（每个棱块有 2 种朝向）。

这一分解是魔方群论分析的基础。需要注意的是，并非 $\mathfrak{S}_8 \times \mathfrak{S}_{12} \times \mathbb{Z}_3^8 \times \mathbb{Z}_2^{12}$ 中的所有元素都对应合法状态——三大不变量（见 §2.4）将合法状态空间缩小为该笛卡尔积的 1/12。

2.2 状态空间与合法状态

若不考虑不变量约束，魔方的”表观状态空间”大小为

$$|\mathfrak{S}_8| \cdot |\mathfrak{S}_{12}| \cdot 3^8 \cdot 2^{12} = 8! \cdot 12! \cdot 3^8 \cdot 2^{12} \approx 5.19 \times 10^{20}.$$

然而，其中只有 1/12 是合法状态。合法状态空间的大小为

$$|\mathcal{R}| = \frac{8! \cdot 12! \cdot 3^8 \cdot 2^{12}}{12} = 43,252,003,274,489,856,000 \approx 4.33 \times 10^{19}. \quad (1)$$

这一数字由 Singmaster 于 1979 年首次严格计算 [1]。1/12 这一因子来自三个独立约束，每个约束将空间缩小为原来的 1/2、1/2、1/3，合计 1/12。

2.3 基本操作与生成元

六个面旋转 U, D, R, L, F, B 是 $\mathcal{R}$ 的标准生成元，但并非最小生成元集。事实上，魔方群可由更少的生成元生成。例如，{U, R, F} 三个面旋转即可生成整个 $\mathcal{R}$ （因为 D 可由 U 与整体翻转复合得到，类似地 L, B 也可导出）。更极端地，甚至存在二元生成元集。

每个生成元的阶为 4（即 $U^4 = e$ ），但 180° 转的阶为 2（如 $U_2^2 = e$ ）。理解生成元的阶与复合行为是分析子群结构（如 Kociemba 算法中的 G_1 子群，见第十章）的前提。

2.4 子群与不变量

不变量是魔方群论的核心。三大不变量将”表观状态空间”缩小为合法状态空间，它们也是判断一个打乱状态是否合法（即是否可通过面旋转从初始状态到达）的充要条件。

定理 2.1 (三大不变量). 设 $g = (\sigma_c, \sigma_e, \vec{v}) \in \mathfrak{S}_8 \times \mathfrak{S}_{12} \times \mathbb{Z}_3^8 \times \mathbb{Z}_2^{12}$ 。g 是合法魔方状态（即 $g \in \mathcal{R}$ ）当且仅当满足以下三个条件：

1. 角块朝向和为零： $\sum_{i=1}^8 v_{c,i} \equiv 0 \pmod{3}$ ；
2. 棱块朝向和为零： $\sum_{j=1}^{12} v_{e,j} \equiv 0 \pmod{2}$ ；
3. 置换奇偶性一致： $\text{sgn}(\sigma_c) = \text{sgn}(\sigma_e)$ ，即角块置换与棱块置换的奇偶性相同。

证明思路. 三个条件的必要性通过验证每个生成元 U, D, R, L, F, B 都保持这些不变量即可 (例如 U 转同时做两个 4-循环, 奇偶性均为偶; F 转改变 4 个角块朝向各 $+1$ 或 -1 , 和为 $0 \pmod 3$)。充分性则需证明: 满足三个条件的任意状态都可通过面旋转从初始状态到达, 这通常通过构造性算法 (如逐块归位) 完成。完整证明见 Frey-Singmaster[2] 第 7-9 章。□

不变量的物理直觉

角块朝向和为零意味着: 若你把一个角块拧歪了 (这在物理上需要拆开魔方), 就无法仅靠面旋转将其单独还原——必须同时拧歪另一个角块以“抵消”朝向偏差。**棱块朝向和为零**同理。**置换奇偶性一致**则意味着: 无法仅靠面旋转交换两个角块而不动其他小块——任何角块交换必须伴随等奇偶性的棱块交换。这三个约束正是“拆开重装”可能产生非法状态的根源。

2.5 奇偶性与约束的推论

不变量定理的一个直接推论是: 魔方群 $\mathcal{R}$ 是 $\mathfrak{S}_8 \times \mathfrak{S}_{12} \times \mathbb{Z}_3^8 \times \mathbb{Z}_2^{12}$ 的一个指数为 12 的子群。这一结构对解法有深远影响:

命题 2.2 (单块操作的不可能性). 以下操作均不属于 $\mathcal{R}$ (即不可能仅靠面旋转实现):

1. 单独翻转一个棱块的朝向 (违反棱块朝向和);
2. 单独旋转一个角块的朝向 (违反角块朝向和);
3. 单独交换两个角块的位置 (违反奇偶性一致性);
4. 单独交换两个棱块的位置 (违反奇偶性一致性)。

这些“不可能操作”恰恰定义了最后一层求解中可能出现的“特殊情况”——例如, CFOP 的 PLL 阶段若遇到单独两个棱块交换, 说明此前某步骤出错或魔方被非法拆装。理解这些约束有助于诊断求解过程中的异常。

2.6 God's Number 与下界

God's Number 定义为: 在最坏情况下, 将任意合法状态还原所需的最少面旋转步数 (每步为一次 90° 或 180° 面转, 180° 计为一步, 即 HTM 度量)。这一数字的确定是魔方数学研究中最具里程碑意义的成果之一。

定理 2.3 (God's Number = 20, HTM 度量). 在 *Half-Turn Metric* (HTM, 180° 转计为一步) 下, 任意合法魔方状态都可在 20 步以内还原, 且存在需要 20 步的状态。因此 *God's Number* = 20。

这一结果由 Rokicki、Kociemba、Davidson 与 Dethridge 于 2010 年通过大规模计算机搜索证明 [11]。证明分为两部分: **上界**——通过对约 5600 万个等价类代表状态逐一验证可在 20 步内还原 (借助 Kociemba 两阶段算法与分布式计算); **下界**——早已知道存在需要 20 步的状态 (如“superflip”状态, 即所有棱块翻转但位置不变)。

God's Number 的确定具有深刻的策略含义: 它告诉我们, **理论上**任何魔方都可在 20 步内还原, 但**实践中**人类记忆与识别能力无法覆盖这一最优路径所需的庞大算法库。这一张力正是各

类解法方法存在的根本原因——它们在”步数效率”与”记忆/识别负担”之间做不同的权衡（详见第十一章）。

3 第一性原理：换位子与共轭

如果说第二章建立了魔方的”静态”结构（群、不变量），那么本章介绍的是”动态”构造工具——换位子与共轭。这两个概念是从第一性原理出发理解**所有**魔方算法的关键：无论是 CFOP 的 OLL/PLL 公式、Roux 的 CMLL、还是盲拧的 3-Style，其算法本质都可以追溯到换位子与共轭的复合。本章是全文的理论枢纽。

3.1 换位子的定义与直觉

定义 3.1 (换位子). 群 G 中元素 a, b 的**换位子**定义为

$$[a, b] = a b a^{-1} b^{-1}.$$

直观上， $[a, b]$ 度量了 a 与 b ”不可交换”的程度：若 a 与 b 交换 ($ab = ba$)，则 $[a, b] = e$ (单位元)；它们越”不交换”， $[a, b]$ 偏离单位元越远。

换位子在魔方中的威力来自一个关键观察：**如果 a 与 b 各自只影响一小部分小块，且这些受影响的小块集合几乎不重叠，那么 $[a, b]$ 只影响这两个集合的”交集”部分。**这是因为 a 先动 A 区、 b 再动 B 区、 a^{-1} 撤销 A 区（但 B 区已被 b 改变，所以 a^{-1} 作用的对象已不同）、 b^{-1} 撤销 B 区——最终只有 $A \cap B$ 中的小块被”真正”改变。

换位子的”交集原理”

设 a 只影响小块集合 A ， b 只影响小块集合 B 。若 $A \cap B$ 非空但 $A \setminus B$ 与 $B \setminus A$ 非空，则换位子 $[a, b]$ 通常只影响 $A \cap B$ 中的小块（位置或朝向），而将 $A \setminus B$ 与 $B \setminus A$ 中的小块恢复原状。这一”交集原理”是构造局部算法的根本机制。

3.2 共轭的作用

定义 3.2 (共轭). 群 G 中，元素 b 经 a 的**共轭**定义为 $a b a^{-1}$ ，记作 b^a 或 ${}^a b$ 。

共轭的几何意义是：”先用 a 把目标位置搬到 b 能作用的地方，执行 b ，再用 a^{-1} 把它搬回去”。在魔方中，共轭是**算法重定位**的核心工具：如果你有一个只在某个特定位置（如 UFR 角）起作用的算法 b ，通过共轭 $a b a^{-1}$ ，你可以让它在任意位置起作用——只需选择适当的 a 将目标位置搬到 UFR。

例 3.1 (共轭重定位). 设 b 是一个只在 UFR 角块上旋转朝向的算法。若你想对 DFL 角块做同样操作，只需取 a 为将 DFL 搬到 UFR 的旋转（如 FL），则 $a b a^{-1}$ 即对 DFL 起作用。这一技巧使得”学一个算法，到处使用”成为可能。

共轭与换位子常常联合使用：先通过共轭将目标小块”对齐”到换位子的作用区，再执行换位子，最后撤销共轭。这一”setup $\rightarrow$ commutator $\rightarrow$ undo setup”模式正是盲拧 3-Style 方法（见 §9.4）的标准结构。

3.3 三循环换位子

魔方求解中最有用的换位子是**三循环**——即只置换三个小块（位置或朝向）而保持其他一切不变的算法。三循环是构造几乎所有高级算法的”原子”。

定理 3.1 (三循环换位子的构造). 设 a 是一个将小块 x 移到位置 p 的操作（且不影响位置 q ）， b 是一个在位置 p 附近起作用的”插入”操作（将某个小块带入或带出 p ）。若 a 与 b 满足交集原理的条件，则 $[a, b] = a b a^{-1} b^{-1}$ 通常实现一个涉及 x 及另外两个小块的三循环。

例 3.2 (角块三循环：A-Perm 的本质). 经典的 A-Perm 算法 $R_2 B' R_2 F_2 L F_2 R' F_2 R_2 F_2$ 实现三个角块的循环置换。从换位子视角看，它可以分解为 $[a, b]$ 的形式，其中 a 是一个将目标角块移入”工作区”的旋转， b 是一个局部插入操作。理解这一分解后，你不再需要”死记”A-Perm——你可以从原理推导它，并通过共轭将其重定位到任意三个角块。

类似地，棱块三循环（如 U-Perm 的本质）、角块朝向三循环（如 Sune 的本质）、棱块朝向三循环（如 $M' U M' U M' U_2 M U M U M U M U_2$ 的本质）都可以从换位子构造出来。这就是为什么有经验的魔方玩家常说：”高级算法不是背出来的，是构造出来的。”

3.4 从换位子构造算法

本节给出一个具体的构造范例，展示如何从第一性原理”设计”一个算法，而非从公式表”查找”它。

目标：构造一个只翻转两个棱块朝向（如 UF 与 UB）的算法。

步骤 1：识别约束。由命题 2.2，单独翻转一个棱块不可能；但翻转两个棱块是合法的（棱块朝向和仍为 $0 \bmod 2$ ）。

步骤 2：选择工作区。我们希望在 U 层操作，以最小化对其他层的干扰。设工作区为 UF-UB 两个棱块。

步骤 3：设计换位子。取 $a = F$ （将 UF 棱块带到 DF 位置，影响 F 面四个棱块）， $b = U' R U R'$ （一个在 R-U 区域翻转棱块朝向的局部操作）。计算 $[a, b] = F (U' R U R') F' (U R' U' R)$ 。通过适当调整，这一换位子可实现 UF 与 UB 两个棱块的翻转。

步骤 4：验证与简化。实际操作中，构造出的算法可能较长，需要通过消去相邻逆元、合并同面旋转等手段简化。简化后的算法即成为可记忆的”公式”。

这一构造过程揭示了所有魔方”公式”的本质：**它们不是任意的旋转串，而是换位子与共轭的精心设计的复合，其”局部性”来自交集原理**。理解这一点后，面对任何陌生算法，你都可以尝试将其分解为换位子结构，从而理解其作用机制而非机械记忆。

3.5 换位子方法的一般框架

综合以上各节，我们可以提炼出从第一性原理构造魔方算法的一般框架：

换位子构造框架

1. **明确目标：**确定要影响的少量小块（通常 2-3 个）及其变化（位置置换或朝向改变）。

2. **验证合法性**: 检查目标操作是否满足三大不变量 (定理 2.1)。若不满足, 目标本身不可能, 需调整。
3. **选择工作区**: 选定一个便于操作的区域 (通常是某一层或某一角), 使目标小块可被“搬入”。
4. **设计 a 与 b** : 找 a (搬运操作, 将目标小块带入工作区) 与 b (局部操作, 在工作区内改变小块)。
5. **计算 $[a, b]$** : 形成换位子, 验证其效果是否为目标。
6. **共轭重定位** (若需要): 用 $g[a, b]g^{-1}$ 将算法重定位到任意位置。
7. **简化与记忆**: 消去冗余、合并旋转, 得到最终算法。

这一框架是第四至十章所有方法的“元算法”。层先法的最后一层算法、CFOP 的 OLL/PLL、Roux 的 CMLL、盲拧的 3-Style——它们在策略层面千差万别, 但在算法构造层面都遵循这一框架。第十一章将系统比较各方法如何在这一框架上做出不同的策略选择。

4 层先法 (LBL): 从零开始的解法

层先法 (Layer-by-Layer, LBL) 是绝大多数魔方初学者接触的第一种系统解法, 也是理解所有进阶方法的基础。它的思想极为朴素——**一层一层地解决**——但其背后蕴含的策略权衡 (先位置后朝向、先底层后顶层、保留已完成部分) 贯穿了几乎所有人类解法。本章从第一性原理出发, 剖析 LBL 的每一步为何如此设计, 而非仅仅罗列步骤。

4.1 思想: 分层解决与“保留已完成”

LBL 的核心策略可以用一句话概括: **从下到上逐层完成, 每完成一层就不再“破坏”它**。这一策略的数学基础是子群思想——每完成一层, 魔方状态就进入一个更小的子群 (该层已固定的状态集合), 后续操作应尽量保持在这一子群内或可逆地离开与返回。

具体而言, LBL 将求解分为三大阶段:

1. **第一层 (底层)**: 完成底十字 (4 个棱块) 与底角 (4 个角块), 使底层四面颜色一致;
2. **第二层 (中层)**: 完成 4 个中层棱块, 使前两层完整;
3. **第三层 (顶层)**: 依次完成顶十字 (棱块朝向)、顶面 (角块朝向)、顶角归位、顶棱归位。

这一分层的精妙之处在于: **每一阶段都只动用尚未完成的部分, 而已完成部分作为“保护区”被有意识地避开**。这与换位子的“交集原理” (§3.1) 一脉相承——LBL 的每一步本质上都是在“未完成区”与“已完成区”的边界上做局部操作, 通过精心设计的旋转序列确保已完成区被恢复。

4.2 第一层：底十字与底角

底十字是 LBL 的第一步，目标是将 4 个含白色（假设白色为底色）的棱块归位到底面，使其侧面颜色与对应中心块对齐。底十字通常不依赖固定算法，而是通过”直觉式”旋转完成——这实际上是在训练玩家对棱块位置与朝向的识别能力。

底十字的关键技巧是”**先对齐侧面，再翻下**”：先将目标棱块的侧面颜色与对应中心块对齐（此时棱块可能在顶层或中层），再用一次旋转将其翻到底层。例如，若白-红棱块在顶层且红色面朝上，先 U 转使红色与红色中心对齐，再 F_2 将其翻到底层。这一过程体现了”先位置后朝向”的子策略。

底角的目标是将 4 个含白色的角块归位到底层四角。每个角块有 3 种朝向，需根据朝向选择不同的插入序列。经典做法是” $R\ U\ R'\ U'$ ”循环——将目标角块置于 UFR 上方，反复执行 $R\ U\ R'\ U'$ 直到角块正确归位。这一循环的数学本质是一个 6 阶操作（执行 6 次回到原状），其每次执行都会改变 UFR 角块的朝向并最终将其”拧入”底层。

底角插入： $R\ U\ R'\ U'$ 循环

目标：将 UFR 上方的角块插入 DFR 底角位置。算法： $R\ U\ R'\ U'$ （重复 1、3 或 5 次，取决于角块朝向）说明：该循环每次执行将 UFR 角块朝向旋转 120° ，同时将其与 DFR 角块交换。重复奇数次后，目标角块以正确朝向落入 DFR，且 U 层其他小块回到原位。

4.3 第二层：中层棱块

第二层的目标是将 4 个不含黄色（假设黄色为顶色）的中层棱块归位。每个棱块需从顶层”插入”到中层左侧或右侧。关键在于：插入操作不能破坏已完成的底层。

经典算法分为”右插”与”左插”两种：

中层棱块：右插与左插

右插（棱块从顶层插入到 FR 位置）： $U\ R\ U'\ R'\ U'\ F'\ U\ F$

左插（棱块从顶层插入到 FL 位置）： $U'\ L'\ U\ L\ U\ F\ U'\ F'$

说明：这两个算法互为镜像。它们的核心是”先让出位置，再插入，再恢复”——通过 U 转让出目标位置，用 R/U 操作将棱块带入，再用 F 操作恢复底层。

从换位子视角看，右插算法 $U\ R\ U'\ R'\ U'\ F'\ U\ F$ 可以分解为两段：前半段 $U\ R\ U'\ R'$ 是一个”取出”操作（将 FR 棱块取出到顶层），后半段 $U'\ F'\ U\ F$ 是一个”插入”操作（将目标棱块插入 FR）。两段合起来恰好完成”取出旧的、插入新的”，且底层在过程中被暂时破坏但最终恢复——这正是换位子”先动后撤”思想的体现。

4.4 第三层：顶十字、顶面、顶角、顶棱

第三层是 LBL 最复杂的部分，通常分为四个子步骤。每个子步骤都依赖固定算法，因为此时前两层已完整，任何操作都必须”在 U 层做文章”且不破坏前两层。

子步骤 1：顶十字 (OLL-Edges)。目标是使 4 个顶棱的黄色面朝上。顶棱朝向有 4 种状态 (0、2-线、2-L、4-十字)，其中”4-十字”已完成。从”点”状态出发，需依次用 $F\ R\ U\ R'\ U'\ F'$ 算法逐步推进：

顶十字算法: $F R U R' U' F'$

点 $\rightarrow$ L 形: $F R U R' U' F'$ (L 形置于左上) L 形 $\rightarrow$ 一字形: $F R U R' U' F'$ (一字形置于水平) 一字形 $\rightarrow$ 十字: $F R U R' U' F'$ 说明: 该算法翻转 UF、UL、UR 三个棱块的朝向 (三循环翻转), 是换位子原理在棱块朝向上的直接应用。

子步骤 2: 顶面 (OLL-Corners)。目标是使 4 个顶角的黄色面朝上。顶角朝向有 7 种状态 (1 种已完成 + 6 种需处理), 每种对应一个算法。最基础的是 Sune 算法 $R U R' U R U^2 R'$, 它实现三个角块的朝向循环:

Sune 算法: $R U R' U R U^2 R'$

作用: 将 UFR、UBR、UBL 三个角块的朝向循环旋转 (每个旋转 120°)。应用: 当顶面有且仅有一个角块黄色朝上时, 将其置于 UBL, 执行 Sune 即可。本质: 角块朝向三循环换位子。

子步骤 3: 顶角归位 (PLL-Corners)。目标是使 4 个顶角到达正确位置 (不管朝向, 朝向已在上一部完成)。此时可能出现三种情况: 已完成、需三循环、需对角交换。三循环用 A-Perm, 对角交换用 E-Perm (或先做一次三循环转化为三循环情况)。

子步骤 4: 顶棱归位 (PLL-Edges)。目标是使 4 个顶棱到达正确位置。此时可能出现三种情况: 已完成、需三循环 (U-Perm)、需对换 (Z-Perm 或 H-Perm)。注意: 由于奇偶性约束 (命题 2.2), 不可能出现”单独两个棱块交换”——若出现, 说明前序步骤有误。

U-Perm (三循环顶棱)

Ua-Perm: $R U' R U R U R U' R' U' R^2$ Ub-Perm: $R^2 U R U R' U' R' U' R' U R'$ 说明: 实现三个顶棱的循环置换, 是棱块三循环换位子的典型实例。

4.5 LBL 的算法表与记忆策略

LBL 第三层共需记忆约 7–10 个算法 (顶十字 1 个 + 顶面 2–7 个 + 顶角 2 个 + 顶棱 2–3 个)。初学者通常先掌握最简版本 (顶面用 Sune 反复执行、顶角用”两角对换”循环、顶棱用 U-Perm 反复执行), 再逐步扩充算法库。

记忆策略上, 建议先理解每个算法的”作用类型” (三循环? 对换? 朝向循环?), 再记忆具体旋转序列。理解作用类型后, 即使忘记具体序列, 也能通过换位子原理重新推导 (参见 §3.4)。

4.6 评价: LBL 的优势与局限

优势: LBL 的最大优势是**概念清晰、入门门槛低**。它不需要大量算法记忆 (最少约 4–6 个算法即可还原), 且每一步的目标明确、可验证。它是所有其他方法的概念基础——理解了 LBL 的”分层 + 保留”策略, 就理解了 CFOP、Roux 等方法的共同骨架。

局限: LBL 的主要局限是**步数效率低** (典型 100–150 步) 与**速度上限低** (熟练后约 30–40 秒)。原因在于: 每一步都严格”先位置后朝向”, 导致大量冗余旋转; 且第三层分四个子步骤, 每个子步骤独立执行, 无法合并。CFOP 正是通过”合并子步骤” (F2L 合并底角与中层棱块、OLL/PLL 合并朝向与位置) 来突破 LBL 的速度上限。

5 CFOP 方法：速度魔方的基石

CFOP (Cross-F2L-OLL-PLL) 是由 Jessica Fridrich 于 1980 年代系统化的层先法进阶版本 [4]，至今仍是全球最流行的速拧方法，几乎所有单手与双手速拧世界纪录均由 CFOP 或其变体创造。CFOP 的核心创新在于将 LBL 的 7 个子步骤合并为 4 个大步骤，通过更大的算法库换取更少的步数与更高的执行流畅度。本章从第一性原理剖析 CFOP 如何在 LBL 的策略基础上实现”并行化”与”算法化”。

5.1 思想：LBL 的并行化与算法化

CFOP 与 LBL 的关系可以用一句话概括：**CFOP 是 LBL 的”合并版”**。具体对应关系如下：

LBL 步骤	CFOP 步骤	合并方式
底十字	Cross	基本不变，但追求更少步数
底角 + 中层棱块	F2L	合并：角块与棱块成对插入
顶十字 + 顶面	OLL	合并：一步完成全部顶面朝向
顶角归位 + 顶棱归位	PLL	合并：一步完成全部顶层位置

每一次”合并”都意味着：用更多的算法（覆盖更多情况）换取更少的步骤数与更少的识别次数。这是 CFOP 速度优势的根本来源——**它用记忆负担换取执行效率**。

从第一性原理看，CFOP 的合并策略是换位子原理的宏观应用：F2L 的”成对插入”本质上是一个同时影响角块与棱块的复合换位子；OLL 的”一步定向”是一个覆盖全部 8 个顶层小块朝向的大换位子；PLL 的”一步归位”是一个覆盖全部 7 个顶层小块位置的大换位子。算法越长，其换位子结构越复杂，但”一次解决”的效率也越高。

5.2 Cross：底十字的优化

CFOP 的 Cross 与 LBL 的底十字目标相同（4 个底棱归位），但追求**更少步数**（理想 8 步以内）与**更优执行**（可盲做、可从任意面开始）。Cross 的优化主要依赖两点：

- 1. 整体规划。**高手在观察阶段（15 秒预观察）会同时考虑 4 个底棱的相对位置，规划一条”一箭双雕”甚至”一箭三雕”的路径，而非逐个解决。这要求对棱块位置与朝向的快速识别能力。
- 2. 灵活朝向。**CFOP 允许从任意面开始做 Cross（不限于白色底），甚至允许在求解过程中翻转整个魔方（虽然现代 CFOP 倾向于减少翻转）。这种灵活性使得 Cross 可以利用更短的旋转路径。

Cross 的”8 步以内”原则

统计表明，约 99% 的打乱状态可在 8 步以内完成 Cross。掌握 Cross 优化的关键不是记忆算法，而是培养”全局视野”——在观察阶段就看到 4 个棱块的相互关系，而非逐个处理。这一能力是 CFOP 与 LBL 最显著的分水岭之一。

5.3 F2L: First Two Layers 的成对插入

F2L 是 CFOP 的核心创新，也是与 LBL 差异最大的部分。LBL 分别解决底角与中层棱块（共 8 步），而 F2L 将每个底角与对应的中层棱块**成对**解决（共 4 对，每对 1 步）。这不仅减少了步骤数，更重要的是减少了”破坏已完成对”的风险。

F2L 的基本思想是：**先将角块与棱块”配对”（在顶层形成一个已配对的角-棱组合），再将配对插入对应槽位。**配对过程在顶层进行，不影响已完成的槽位；插入过程用 $R\ U\ R'$ 或 $L'\ U'\ L$ 等短序列完成。

F2L 共有 41 种标准情况（4 个槽位 $\times$ 约 10 种情况/槽位），但本质上只有 3 大类：

F2L 的三大类情况

1. **角块与棱块均在顶层**（最常见，约 27 种）：先通过 U 转与 R/U 操作将二者配对，再插入。关键技巧是”隐藏”——将一个块暂时移到非工作面，为另一个块腾出配对空间。
2. **角块在槽位、棱块在顶层**（约 8 种）：先将角块”取出”到顶层（同时可能改变其朝向），再按第一类处理。
3. **棱块在槽位、角块在顶层**（约 6 种）：类似第二类，先取出棱块。

F2L 基本插入： $R\ U\ R'$ （最简配对插入）

情况：角块在 UFR 上方，棱块在 UF ，二者已”配对”（颜色匹配）。算法： $R\ U\ R'$ 说明：将配对直接插入 FR 槽。这是 F2L 最短的插入序列（3 步）。其镜像为 $L'\ U'\ L$ （插入 FL 槽）。

F2L 的学习曲线较陡，但掌握后速度提升显著。高级 F2L 技巧包括：**多槽位预判**（在插入当前对的同时观察下一对的起始位置）、**空槽利用**（利用尚未填充的槽位作为”中转站”简化配对）、**ZBLS/ZBF2L**（将 F2L 与 OLL 部分合并，共 300+ 算法）。

5.4 OLL: Orient Last Layer 的一步定向

OLL 的目标是**一步**使全部 8 个顶层小块（4 棱 + 4 角）的黄色面朝上。LBL 分两步（顶十字 + 顶面）完成，CFOP 合并为一步。OLL 共有 57 种情况（1 种已完成 + 56 种需处理），是 CFOP 算法量最大的部分。

OLL 的 57 种情况可按”顶十字是否已形成”分为两大类：

- **有十字类**（27 种）：顶棱已定向，只需处理顶角。包括 Sune、Anti-Sune、Headlights、Chameleon、Bowtie、Pi、H 等子类。
- **无十字类**（30 种）：顶棱未全部定向，需同时处理棱与角。包括”点”、”L”、”线”等起始形态及其与角块朝向的组合。

Sune (OLL 27): $R\ U\ R'\ U\ R\ U^2\ R'$

情况：顶面有 1 个角块黄色朝上（置于 UBL ），顶十字已形成。作用：循环旋转 UFR 、 UBR 、 UBL 三个角块的朝向。本质：角块朝向三循环换位子（见 S3.3）。

OLL 21 (H 形): $R\ U^2\ R'\ U'\ R\ U\ R'\ U'\ R\ U'\ R'$

情况：顶十字已形成，4 个角块黄色均不朝上，呈”H”对称形态。作用：一步定向全部 4 个角块。

OLL 的学习通常分阶段：先学 2-look OLL（顶十字 + 顶面，共 7-10 个算法，即 LBL 的做法），再逐步扩充到 full OLL（57 个）。2-look OLL 是 LBL 到 CFOP 的自然过渡。

5.5 PLL: Permute Last Layer 的一步归位

PLL 的目标是**一步**使全部 7 个顶层小块（4 棱 + 4 角，中心固定）到达正确位置。此时朝向已全部正确（OLL 已完成），只需置换。PLL 共有 21 种情况（1 种已完成 + 21 种需处理），按作用类型分为：

类型	算法数	代表
角块三循环（A-Perm）	2	Aa, Ab
棱块三循环（U-Perm）	2	Ua, Ub
角块 + 棱块混合三循环（J, T, F, R）	8	Jb, T, F, R
双对换（H, Z, E）	3	H-Perm, Z-Perm, E-Perm
四循环（G-Perm）	4	Ga, Gb, Gc, Gd
角块对换 + 棱块三循环（N, V, Y）	4	Na, Nb, V, Y

T-Perm（最常用 PLL 之一）： $R\ U\ R'\ U'\ R'\ F\ R^2\ U'\ R'\ U'\ R\ U\ R'\ F'$

情况：相邻两角对换 + 相邻两棱对换（呈 "T" 形侧面图案）。作用：同时交换 $UFR\leftrightarrow UBR$ 两角与 $UF\leftrightarrow UR$ 两棱。本质：两个对换的复合，满足奇偶性约束（偶置换）。

H-Perm： $M^2\ U\ M^2\ U^2\ M^2\ U\ M^2$

情况：两对对角棱分别对换（ $UF\leftrightarrow UB$, $UR\leftrightarrow UL$ ）。作用：实现两个不相邻棱对换。说明：使用 **M** 中层转，步数少且执行流畅，是 PLL 中最优雅算法之一。

PLL 的学习也分阶段：先学 2-look PLL（先角后棱，共 6 个算法），再扩充到 full PLL（21 个）。

5.6 算法量与学习曲线

Full CFOP 的总算法量约为：Cross（0，靠直觉）+ F2L（41）+ OLL（57）+ PLL（21）= **约 119 个算法**。这是一个相当大的记忆负担，但通过分阶段学习（2-look → full）可以平滑过渡：

阶段	算法数	预期成绩
Beginner LBL	4-6	60-90 秒
2-look CFOP	16	25-35 秒
Full CFOP	119	12-20 秒
CFOP + 高级技巧	119+	sub-10 秒

5.7 评价：CFOP 的优势与局限

优势：CFOP 的最大优势是**速度上限高**（世界纪录 sub-5 秒）与**生态成熟**（算法库、教程、训练方法极为完善）。其“分层 + 合并”的策略使得每一步都有明确目标，且步骤间过渡流畅。此外，CFOP 的“高算法量”在熟练后反而成为优势——更多算法意味着更少的步数与更少的识别停顿。

局限：CFOP 的主要局限是**学习曲线陡**（full CFOP 需数月记忆）与**旋转多**（F2L 阶段频繁的 R/U 转导致整体旋转量大，影响单手速度）。此外，CFOP 的“严格分层”策略在某些状态下并非最优——例如，当顶层已部分定向时，强制走 Cross→F2L→OLL→PLL 可能比 Roux 的灵活块构建更慢。这些局限正是 Roux、ZZ 等方法存在的理由。

6 Roux 方法：块构建的艺术

Roux 方法由法国魔方玩家 Gilles Roux 于 2003 年提出 [5]，是 CFOP 之外最具影响力的速拧方法之一。与 CFOP 的“分层”策略不同，Roux 采用“**块构建**”（**blockbuilding**）策略——先在魔方两侧构建两个 $1 \times 2 \times 3$ 的长方块，再以极少旋转完成剩余部分。Roux 的标志性特征是**几乎不需要整体旋转**（**cube rotation**）且大量使用 **M 中层转**，这使其在单手速拧（one-handed）领域具有独特优势。本章从第一性原理剖析 Roux 的块构建策略及其与 CFOP 的根本差异。

6.1 思想：块构建与桥式结构

Roux 的核心策略可以概括为“**两侧建桥，中间收尾**”：先在魔方左下与右下各构建一个 $1 \times 2 \times 3$ 的长方块（各含 1 个角块 + 2 个棱块 + 1 个中层棱块，共 6 个小块），这两个“桥”完成后，魔方剩余 6 个棱块（顶层 4 个 + 中层 2 个）与 4 个顶层角块未解决。此后，Roux 用 CMLL（角块定向与归位）与 LSE（最后六棱）两步收尾。

Roux 的四阶段结构

1. **First Block (FB)**：构建左下 $1 \times 2 \times 3$ 块（约 5–10 步）；
2. **Second Block (SB)**：构建右下 $1 \times 2 \times 3$ 块（约 5–10 步）；
3. **CMLL**：定向并归位 4 个顶层角块（约 9–12 步，42 个算法）；
4. **LSE (Last Six Edges)**：解决最后 6 个棱块（约 8–12 步，无固定算法，靠直觉与模式）。

Roux 与 CFOP 的根本差异在于“**块**”与“**层**”的区别。CFOP 的 F2L 构建的是完整的两层（ $2 \times 3 \times 3$ ），而 Roux 的两个 Block 构建的是两个 $1 \times 2 \times 3$ 的“半层”，中间留出 M 层通道。这一差异使得 Roux 在最后阶段可以用 M 转高效地解决剩余棱块，而无需像 CFOP 那样依赖大量 R/U 转。

从第一性原理看，Roux 的块构建策略是对“保留已完成”原理的更灵活运用：CFOP 严格保留“层”，而 Roux 保留“块”——块比层更灵活，可以在任意位置构建，且构建过程可以利用 M 层的“自由度”做更高效的配对。

6.2 First Block (第一块)

First Block 的目标是在左下构建一个 $1 \times 2 \times 3$ 块，通常以白-绿-红（或任意选定颜色组合）为基准。该块包含：DLF 角块 + DL 棱块 + FL 棱块 + DBL 角块 + BL 棱块 + DF 棱块（共 1 角 + 2 棱 + 1 角 + 2 棱 = 2 角 + 4 棱，但实际是 2 角 + 4 棱中的一部分，构成 $1 \times 2 \times 3$ ）。

FB 的构建高度依赖直觉与规划，没有固定算法。关键技巧是”先角后棱，逐步扩展”：先归位一个角块作为”锚点”，再围绕它逐步添加棱块，形成 $1 \times 1 \times 2 \rightarrow 1 \times 1 \times 3 \rightarrow 1 \times 2 \times 3$ 的扩展序列。高手通常在 15 秒预观察内规划 FB 的完整路径，目标 5-7 步完成。

FB 的”自由度”优势

与 CFOP 的 Cross 不同，Roux 的 FB 只需固定一个 $1 \times 2 \times 3$ 区域，其余 5 个面完全自由。这意味着 FB 可以利用更多”捷径”——例如，当一个棱块恰好在目标位置附近时，只需 1-2 步即可归位，而 CFOP 的 Cross 可能需要更多调整。这一”自由度”是 Roux 步数效率的来源之一。

6.3 Second Block (第二块)

Second Block 的目标是在右下构建另一个 $1 \times 2 \times 3$ 块，与 FB 对称。SB 的构建比 FB 更具挑战性，因为此时 FB 已完成，SB 的构建不能破坏 FB。关键策略是”只用 R、U、M 三面”——通过限制操作面，确保 FB 不被触及。

SB 的典型构建方式是：先用 U 转将目标小块对齐，再用 R 或 M 转将其”插入”右下区域。由于 M 转同时影响左右两侧，需谨慎使用以避免破坏 FB。高级技巧包括”M-unlock”（用 M 转暂时”解锁”FB 的一部分以简化 SB 构建，再恢复）。

6.4 CMLL: Corners of Last Layer

CMLL 的目标是**一步**定向并归位 4 个顶层角块。此时两个 Block 已完成，M 层与 U 层尚未解决。CMLL 共有 42 种情况（与 CFOP 的 OLL+PLL 角块部分相同），但 Roux 的 CMLL 算法与 CFOP 的 CLS/PLL 角块算法不同——Roux 的 CMLL 允许破坏 M 层（因为 M 层尚未解决），因此算法可以更短、更灵活。

CMLL 示例：Sune（与 CFOP 的 Sune 相同）

算法：R U R' U R U2 R' 说明：在 Roux 中，该算法不仅定向角块，还可能打乱 M 层——但这不影响 Roux 流程，因为 M 层将在 LSE 阶段解决。这一”允许破坏 M 层”的自由度使 Roux 的 CMLL 算法比 CFOP 的对应算法更短。

CMLL 的 42 个算法可分为 6 大类：Sune、Anti-Sune、H、Pi、L、T、U（与 OLL 的角块部分类似）。学习 CMLL 的关键是识别角块朝向与位置的组合模式。

6.5 LSE: Last Six Edges

LSE 是 Roux 最具特色的阶段，目标是解决最后 6 个棱块（顶层 4 个 + 中层左右 2 个）。LSE **没有固定算法**，而是依赖对 M 层与 U 层模式的理解，通过 M 与 U 转的组合高效完成。这

是 Roux 与 CFOP 最显著的差异——CFOP 的最后阶段依赖大量记忆算法，而 Roux 的 LSE 依赖直觉与模式识别。

LSE 通常分为 3 个子步骤：

LSE 的三子步骤

1. **Edge Orientation (EO)**：定向 6 个棱块（使顶层棱块朝向正确）。通过 M 转与 U 转的组合，将”坏棱块”（朝向错误）逐步消除。典型 2–4 步。
2. **UL/UR 归位**：将 UL 与 UR 两个棱块归位（作为”锚点”）。通过 M 转将它们对齐到正确位置。典型 1–3 步。
3. **剩余 4 棱归位**：通过 $M^2 U M^2 U^2 M^2 U M^2$ （H-Perm 变体）或类似模式解决剩余 4 个棱块。典型 4–6 步。

LSE 经典收尾： $M^2 U M^2 U^2 M^2 U M^2$

情况：6 个棱块已定向，UL/UR 已归位，剩余 4 个顶层棱块需两两对换。算法： $M^2 U M^2 U^2 M^2 U M^2$ 说明：即 H-Perm，但在 Roux 中它同时解决顶层 4 棱与中层 2 棱的最终归位。

LSE 的”无算法”特性使其成为 Roux 学习曲线中最难的部分，但也是最具创造性的部分。熟练后，LSE 可以根据具体情况即兴构造最优路径，而非机械执行固定算法。

6.6 评价：Roux 的优势与局限

优势：Roux 的最大优势是**步数效率高**（典型 45–55 步，比 CFOP 少 10–15 步）与**旋转极少**（几乎不需要整体旋转，单手友好）。此外，Roux 的 LSE 阶段依赖直觉而非记忆，使得高级 Roux 玩家在面对新状态时更具适应性。Roux 在单手速拧（OH）领域尤为强势，多个 OH 世界纪录由 Roux 创造。

局限：Roux 的主要局限是**学习曲线陡峭且非主流**（教程与社区资源少于 CFOP）、**M 转要求高**（M 转的执行速度与精度直接影响成绩，而 M 转比 R/U 转更难快速执行）、**FB/SB 规划难**（块构建的直觉需要大量练习培养）。此外，Roux 的”两侧建桥”策略在某些打乱状态下可能比 CFOP 的 Cross 更慢——当底棱分散且难以快速配对时，FB 的规划成本较高。

7 ZZ 方法：无旋转的优雅

ZZ 方法由波兰魔方玩家 Zbigniew Zborowski 于 2006 年提出 [6]，其核心创新是在**求解最开始就完成所有棱块的定向（Edge Orientation, EO）**，此后整个求解过程**不需要任何整体旋转（rotationless）**，只需 R、U、L、D、F、B 六面转（且 F/B 只用 180° ）。这一特性使 ZZ 在执行流畅度与人体工学方面具有独特优势。本章从第一性原理剖析 ZZ 的”EO 优先”策略及其深远影响。

7.1 思想：边定向预处理

ZZ 的核心洞察是：**棱块的朝向（orientation）是影响后续操作自由度的关键因素**。如果所有棱块都已定向（即”好棱块”），那么后续只需用 $\langle R, L, U, D, F^2, B^2 \rangle$ 这组”保持定向”的旋转即

可完成求解，无需任何整体旋转。这一“EO 优先”策略将“定向问题”与“位置问题”彻底分离，使求解过程更加可控。

ZZ 的三阶段结构

1. **EOLine**：定向全部 12 个棱块 + 完成 DF 与 DB 两个棱块（形成“线”），约 5–8 步；
2. **ZZ-F2L**：完成前两层（类似 CFOP 的 F2L，但无旋转），约 4 个块 $\times$ 5–7 步；
3. **LL (Last Layer)**：完成最后一层，可用 OLL/PLL（与 CFOP 相同）或更高级的 ZBLL（一步完成 LL，共 493 个算法）。

ZZ 与 CFOP/Roux 的根本差异在于“定向优先”：CFOP 与 Roux 在求解过程中逐步处理定向（CFOP 的 OLL、Roux 的 LSE-EO），而 ZZ 在最开始就一次性解决所有棱块定向。这一“前置”策略的代价是 EOLine 的规划难度较高，但收益是后续全程无旋转、无定向干扰。

从第一性原理看，ZZ 的 EO 策略是对“不变量原理” (§2.4) 的主动利用：棱块朝向和为零这一不变量意味着，只要所有棱块定向，后续操作只要不破坏这一定向（即只用 R, L, U, D, F₂, B₂），就永远保持在“全定向”子群内。这一子群结构是 ZZ 无旋转特性的数学基础。

7.2 EOLine：边定向 + 线

EOLine 是 ZZ 最关键也最难阶段，目标有二：(1) **定向全部 12 个棱块**（使所有棱块的“好/坏”属性变为“好”）；(2) **完成 DF 与 DB 两个棱块**（形成一条竖线）。EOLine 通常需 5–8 步，但规划难度高，需在 15 秒预观察内完成。

棱块定向的判定：一个棱块是“好”（good）还是“坏”（bad）取决于其颜色与 U/D 面的关系。简言之，若棱块含 U/D 色（白/黄），则该色朝 U/D 为“好”；若棱块不含 U/D 色（中层棱块），则其 F/B 色不朝 F/B 为“好”。“坏”棱块需通过 F、B、R、L 的 90° 转来翻转（U、D 不影响定向，F₂、B₂、R₂、L₂ 也不影响）。

EO 的执行：识别所有“坏”棱块（通常 4–8 个），通过 F/B 转（每次翻转 4 个棱块的定向）将它们逐步转为“好”。关键技巧是“成对翻转”——由于棱块朝向和为偶数（不变量），坏棱块数必为偶数，可成对处理。

EOLine 的“线”为何重要

EOLine 不仅完成 EO，还构建 DF–DB 这条“线”作为后续 F2L 的“锚点”。这条线相当于 CFOP 的 Cross 的简化版（只 2 个棱块），但它与 EO 结合后，使得 F2L 阶段可以无旋转地从两侧扩展。这一“线 + EO”的组合是 ZZ 独有的设计。

7.3 ZZ-F2L：无旋转的前两层

EOLine 完成后，ZZ-F2L 从 DF–DB 线向两侧扩展，构建前两层。与 CFOP 的 F2L 类似，ZZ-F2L 也是成对插入角块与棱块，但有关键差异：**全程只用 R, L, U, D（及 F₂, B₂），无 90° 的 F/B 转，无整体旋转。**

这一限制的数学基础是：R, L, U, D, F₂, B₂ 这组旋转**保持棱块定向**（不改变任何棱块的“好/坏”属性）。因此，ZZ-F2L 在“全定向”子群内进行，不会破坏 EOLine 的成果。

ZZ-F2L 基本插入（无旋转版）

情况：角块在 UFR，棱块在 UF，已配对。算法： $R\ U\ R'$ （与 CFOP 相同，但 ZZ 中此操作不破坏 EO）说明：由于 EO 已完成， $R\ U\ R'$ 不会产生新的“坏”棱块。ZZ-F2L 的算法与 CFOP 的 F2L 算法高度重叠，但 ZZ 玩家需避免使用含 F/B 90° 转的算法。

ZZ-F2L 的 4 个块从下到上、从内到外依次构建。由于无旋转，ZZ-F2L 的执行极为流畅，特别适合追求高 TPS（turns per second）的玩家。

7.4 LL：最后一层

ZZ 的最后一层有多种选择，从简到繁：

- 1. **OLL/PLL（与 CFOP 相同）**：由于 EO 已完成，ZZ 的 OLL 只需处理角块定向（27 种，而非 CFOP 的 57 种），PLL 完全相同（21 种）。这是 ZZ 最简单的 LL 路径，共 48 个算法。
- 2. **ZBLL（Zborowski-Bruchem Last Layer）**：由于 EO 已完成，ZZ 可以一步完成整个 LL（定向 + 归位），共 493 个算法。这是魔方所有方法中算法量最大的 LL 系统，但掌握后 LL 步数极少（平均 12–15 步）。
- 3. **COLL/EPLL**：折中方案，COLL 一步定向并归位角块（40 个算法），EPLL 解决棱块归位（4 个算法，因角块已归位，EPLL 只有 4 种情况）。共 44 个算法，是 OLL/PLL 与 ZBLL 之间的平衡点。

ZZ LL 的算法量对比		
LL 方法	算法数	平均 LL 步数
OLL/PLL（ZZ 版）	48	15–18
COLL/EPLL	44	12–15
ZBLL	493	8–12

注：ZZ 的 OLL 只有 27 种（因 EO 已完成），比 CFOP 的 57 种少一半。

7.5 评价：ZZ 的优势与局限

优势：ZZ 的最大优势是**无旋转**（全程不需整体旋转，执行极流畅）、**人体工学好**（只用 R/U/L/D，手指负担小）、**LL 算法少**（因 EO 已完成，OLL 减半）。此外，ZZ 的“EO 优先”策略使后续阶段无定向干扰，求解过程更加“干净”。ZZ 在盲拧预处理（EO 作为盲拧的第一步）与少步数赛（fewest moves）中也有应用。

局限：ZZ 的主要局限是 **EOLine 规划难**（识别坏棱块与规划翻转路径需要大量练习，15 秒预观察往往不够）、**F2L 灵活性受限**（无 F/B 90° 转的限制使得某些 F2L 情况需更多步数）、**非主流**（社区资源与教程少于 CFOP）。此外，ZZ 的“EO 优先”策略在某些打乱状态下可能需较多步数（当坏棱块分散时），不如 CFOP 的 Cross 灵活。

8 Petrus 方法：最小破坏的策略

Petrus 方法由瑞典魔方玩家 Lars Petrus 于 1980 年代提出 [7]，是最早的”块构建”方法之一，也是少步数赛 (fewest moves) 中的常用方法。Petrus 的核心策略是”**逐步扩展块 + 尽早完成棱块定向**”，通过最小化对已完成部分的破坏来提高效率。本章从第一性原理剖析 Petrus 的”最小破坏”策略及其与 Roux、ZZ 的异同。

8.1 思想：逐步构建与保留定向

Petrus 的核心策略可以概括为”**从小到大逐步构建块，并在块构建过程中尽早完成棱块定向**”。与 Roux 的”两侧建桥”不同，Petrus 从一个 $2 \times 2 \times 2$ 角块开始，逐步扩展到 $2 \times 2 \times 3$ 、 $3 \times 2 \times 3$ (前两层)，最后解决顶层。关键创新是在 $2 \times 2 \times 3$ 阶段后就**完成所有棱块定向**，此后只需用保持定向的旋转 (R, L, U, D, F₂, B₂) 完成剩余部分。

Petrus 的阶段结构

1. $2 \times 2 \times 2$ 块：构建一个角块周围的 $2 \times 2 \times 2$ 立方体 (1 角 + 3 棱)，约 3-6 步；
2. 扩展到 $2 \times 2 \times 3$ ：将 $2 \times 2 \times 2$ 扩展为 $2 \times 2 \times 3$ 长方体 (+1 角 +2 棱)，约 3-5 步；
3. 棱块定向 (EO)：定向剩余所有棱块，约 2-4 步；
4. 完成 F2L：扩展到前两层 ($3 \times 2 \times 3$)，约 4-6 步；
5. LL (Last Layer)：解决最后一层，可用 OLL/PLL 或更高级的 LL 方法。

Petrus 与 Roux 的根本差异在于**块的形状与位置**：Roux 构建两个分离的 $1 \times 2 \times 3$ 块 (左右两侧)，而 Petrus 构建一个连续扩展的块 (从一个角开始)。Petrus 与 ZZ 的共同点是**尽早完成 EO**，但 Petrus 在 $2 \times 2 \times 3$ 后做 EO (比 ZZ 的 EOLine 更晚)，灵活性更高。

从第一性原理看，Petrus 的”最小破坏”策略是对”保留已完成”原理的极致运用：每一步都只扩展已完成的块，且扩展操作尽量不破坏已有部分。这一策略使得 Petrus 的步数效率极高 (典型 45-55 步)，是少步数赛的常用方法。

8.2 $2 \times 2 \times 2$ 块

$2 \times 2 \times 2$ 块是 Petrus 的起点，目标是在一个角块周围构建一个 $2 \times 2 \times 2$ 立方体 (包含 1 个角块 + 3 个棱块 + 0 个角块，实际上是 1 角 + 3 棱，构成一个 $2 \times 2 \times 2$ 区域)。通常选择 DBL (下后左) 角作为起点，构建包含 DBL 角、DL 棱、BL 棱、DB 棱的 $2 \times 2 \times 2$ 块。

构建 $2 \times 2 \times 2$ 块高度依赖直觉，没有固定算法。关键技巧是”**先角后棱，逐步包围**”：先归位 DBL 角块，再依次归位 DL、BL、DB 三个棱块，形成 $2 \times 2 \times 2$ 。高手通常在 15 秒预观察内规划 $2 \times 2 \times 2$ 的路径，目标 3-5 步完成。

$2 \times 2 \times 2$ 的”自由度”优势

$2 \times 2 \times 2$ 块只固定一个角区域，其余 7 个角完全自由。这使得 $2 \times 2 \times 2$ 的构建比 CFOP 的 Cross (固定 4 个棱) 或 Roux 的 FB (固定 6 个小块) 更灵活。Petrus 的”从小到大”策略使得每一步的”自由度”都最大化，这是其步数效率的来源。

8.3 扩展到 $2 \times 2 \times 3$

$2 \times 2 \times 2$ 完成后，Petrus 将其扩展为 $2 \times 2 \times 3$ 长方体。这一步添加 1 个角块 (DBR) 与 2 个棱块 (DR、BR)，使块从 $2 \times 2 \times 2$ 扩展为 $2 \times 2 \times 3$ 。扩展操作需保持 $2 \times 2 \times 2$ 不被破坏，通常只用 R,U 两面转（假设 $2 \times 2 \times 2$ 在 DBL 位置）。

8.4 棱块定向 (EO)

$2 \times 2 \times 3$ 完成后，Petrus 的关键创新是**立即完成所有剩余棱块的定向**。此时已有 6 个棱块在 $2 \times 2 \times 3$ 块中（已定向），剩余 6 个棱块（顶层 4 个 + 中层 2 个）需定向。EO 通过 F/B 的 90° 转完成（每次翻转 4 个棱块），通常 2-4 步。

Petrus EO 与 ZZ EO 的对比

Petrus 的 EO 在 $2 \times 2 \times 3$ 后进行，此时已有 6 个棱块固定，只需定向剩余 6 个——比 ZZ 的 EOLine（定向全部 12 个棱块）更简单。但 Petrus 的 EO 需保持 $2 \times 2 \times 3$ 块不被破坏，限制了操作自由度。这一权衡是 Petrus 与 ZZ 的关键差异。

8.5 完成 F2L 与 LL

EO 完成后，Petrus 将 $2 \times 2 \times 3$ 扩展为完整的前两层 ($3 \times 2 \times 3$)，添加 1 个角块 (DFR) 与 2 个棱块 (FR、FL)。此后只用保持定向的旋转 (R,L,U,D,F₂,B₂) 完成。

LL 阶段，Petrus 与 ZZ 类似，因 EO 已完成，OLL 只需处理角块定向 (27 种)。Petrus 常用 COLL/EPLL 或 ZBLL 作为 LL 方法。由于 Petrus 的 F2L 完成后，顶层角块与棱块的位置关系可能比 CFOP 更“乱”（因为 Petrus 的 F2L 不强制顶层状态），LL 的识别可能更复杂。

8.6 评价：Petrus 的优势与局限

优势：Petrus 的最大优势是**步数效率极高**（典型 45-55 步，是少步数赛的常用方法）、**EO 早期完成**（后续无定向干扰，与 ZZ 类似）、**块构建灵活**（从小到大逐步扩展，自由度高）。此外，Petrus 的“最小破坏”策略使得每一步都高效，冗余旋转少。

局限：Petrus 的主要局限是**速度上限较低**（因块构建依赖直觉，TPS 难以提升，典型 15-25 秒，不如 CFOP 的 sub-10）、**非主流**（社区资源少，学习者少）、**EO 识别难**（在 $2 \times 2 \times 3$ 后识别坏棱块并规划翻转路径需要大量练习）。Petrus 更多用于少步数赛与作为理解块构建原理的教学方法，而非主流速拧方法。

9 盲拧方法：序列化与缓冲区

盲拧 (Blindfolded Solving, 简称 BLD) 是魔方竞速中最具挑战性的项目之一：选手在观察阶段记忆魔方状态，蒙眼后凭记忆还原。盲拧方法的设计哲学与速拧方法截然不同——速拧追求“看到即还原”的快速识别-执行循环，而盲拧追求“**先编码为序列，再机械执行**”的可靠性。本章介绍三种主流盲拧方法：Old Pochmann、M2、3-Style，它们分别代表了盲拧从“单块交换”到“三循环”的演进，其背后正是换位子原理（第三章）的逐步深化。

9.1 思想：将求解转化为序列执行

盲拧的核心思想可以概括为“缓冲区 + 循环分解”：选定一个“缓冲位置” (buffer)，将魔方的打乱状态分解为一系列“从缓冲区出发、经过若干目标、回到缓冲区”的循环。每个循环中的每一步对应一个“将缓冲区的小块送到目标位置”的算法。执行完所有算法后，魔方还原。

这一思想的关键洞察是：盲拧不需要“看到”整个魔方的变化，只需按预先编码的序列执行**固定算法**。每个算法只影响缓冲区与目标位置（及其“轨道”上的少数小块），且算法之间互不干扰（通过精心设计确保）。这使得盲拧可以“闭眼”执行——只要序列正确，结果必然正确。

盲拧的“缓冲区-循环”模型

设缓冲位置为 B 。魔方的打乱状态可分解为若干置换循环：

$$(B a_1 a_2 \cdots a_k)(B b_1 b_2 \cdots b_m) \cdots$$

每个循环 $(B a_1 \cdots a_k)$ 表示“ B 的小块应去 a_1 ， a_1 的小块应去 a_2 ， $\cdots$ ， a_k 的小块应去 B ”。盲拧通过依次执行“将 B 送到 a_1 ”、“将 B 送到 a_2 ”、 $\cdots$ 的算法来还原每个循环。

从第一性原理看，盲拧的“缓冲区-循环”模型是换位子原理的宏观应用：每个“将 B 送到目标”的算法本质上是一个**换位子**（或共轭的换位子），它只影响 B 与目标位置（及少量“轨道”小块）。Old Pochmann 用“对换” (2-cycle)，M2 用“M 层对换”，3-Style 用“三循环”——算法的“作用范围”越来越大，效率越来越高，但对记忆与识别的要求也越来越高。

9.2 Old Pochmann：经典对换方法

Old Pochmann (OP) 由 Stefan Pochmann 提出 [8]，是最早的系统盲拧方法，也是大多数盲拧初学者的入门方法。OP 的核心是用“**对换**” (2-cycle) **逐个解决小块**——每次将缓冲区的小块与目标位置的小块交换，直到所有小块归位。

角块 OP：缓冲区为 UBL 角。使用 Y-Perm 算法 ($R U' R' U' R U R' F' R U R' U' R' F R$) 作为“交换算法”，通过共轭 (setup moves) 将目标角块搬到 Y-Perm 的作用位置，执行 Y-Perm，再撤销 setup。每次执行交换 UBL 与目标角块（同时交换两个棱块作为“副作用”，但棱块单独处理）。

棱块 OP：缓冲区为 UB 棱。使用 T-Perm 算法 ($R U R' U' R' F R^2 U' R' U' R U R' F'$) 作为“交换算法”，通过 setup 将目标棱块搬到 T-Perm 的作用位置，执行 T-Perm，再撤销 setup。每次执行交换 UB 与目标棱块（同时交换两个角块作为“副作用”，但角块单独处理）。

Old Pochmann 棱块交换 (T-Perm + setup)

目标：交换 UB (缓冲) 与目标棱块 X 。步骤：1. **Setup**：用 U/D/R/L/F/B 转将 X 搬到 UF 位置 (T-Perm 的作用位置)。2. 执行 T-Perm: $R U R' U' R' F R^2 U' R' U' R U R' F'$ 3. **撤销 Setup**：逆序执行步骤 1 的逆操作。说明：T-Perm 交换 $UB \leftrightarrow UF$ 两棱（及 $UFR \leftrightarrow UBR$ 两角作为副作用）。角块副作用在角块 OP 阶段被“抵消”（因角块与棱块分别处理）。

OP 的优点是**算法量少**（只需记忆 Y-Perm 与 T-Perm 两个核心算法 + 各位置的 setup）与**可靠性高**（每次只做一个对换，不易出错）。缺点是**步数多**（每个小块需一次完整算法，典型 200+ 步）与**速度慢**（典型 1-2 分钟）。

9.3 M2 方法：M 层对换

M2 方法是 OP 棱块部分的高级替代，由多个盲拧玩家发展完善。M2 的核心是用 **M2 转 (中层 180°)** 作为棱块交换的核心操作，通过 setup 将目标棱块搬到 M 层，执行 M2，再撤销 setup。M2 比 OP 的 T-Perm 更短 (1 步 vs 14 步)，大幅提升棱块盲拧效率。

M2 棱块交换

目标：交换 DF (缓冲) 与目标棱块 X 。步骤：1. **Setup**：将 X 搬到 DB 位置 (M2 的对位)。2. 执行 M2 (中层 180° 转，交换 $DF \leftrightarrow DB$)。3. 撤销 **Setup**。说明：M2 只交换 DF 与 DB 两个棱块，无角块副作用。但 M2 会改变 M 层其他棱块的位置 (奇偶性问题)，需通过“奇偶性算法”处理。

M2 的角块部分仍用 OP (Y-Perm)，因此 M2 方法实际上是“M2 棱块 + OP 角块”的组合。M2 的优点是**棱块步数大幅减少** (每个棱块约 3-5 步 vs OP 的 14 步) 与**执行更快** (典型 40-60 秒)。缺点是 **setup 复杂** (不同目标位置需不同 setup，需记忆 12 个 setup 序列) 与**奇偶性处理** (当棱块循环数为奇数时需额外算法)。

9.4 3-Style：换位子方法的高级应用

3-Style 是盲拧的最高级方法，其核心是用**三循环换位子 (3-cycle commutator)** 一次解决三个小块，而非 OP/M2 的“一次一个对换”。3-Style 直接应用第三章的换位子原理：每个算法是一个精心设计的换位子 $[a, b]$ ，实现缓冲区与两个目标位置的三循环。

3-Style 的换位子结构

3-Style 的每个算法形如：

$$\text{Setup} \rightarrow [a, b] = a b a^{-1} b^{-1} \rightarrow \text{Undo Setup}$$

其中 $[a, b]$ 实现缓冲区 B 与两个目标 X, Y 的三循环 (BXY)。通过选择不同的 a (搬运) 与 b (插入)，可以构造覆盖所有目标对的算法。

3-Style 的算法量极大：角块约 378 个 (21 个目标位置 $\times$ 18 个配对)，棱块约 462 个。但每个算法通常只有 8-12 步 (远短于 OP 的 14 步)，且一次解决三个小块 (效率是 OP 的 3 倍)。3-Style 的优点是**步数极少** (典型 60-80 步) 与**速度极快** (顶级选手 sub-20 秒)。缺点是**算法量巨大** (需记忆 800+ 算法) 与**学习曲线极陡** (需数月甚至数年)。

3-Style 与换位子原理的关系

3-Style 是第三章换位子原理的最纯粹应用。每个 3-Style 算法都可以从第一性原理推导：选定缓冲区 B 与两个目标 X, Y ，找一个将 B 搬到 X 附近的操作 a ，与一个在 X 附近插入 Y 的操作 b ，形成 $[a, b]$ 。理解这一构造后，3-Style 的 800+ 算法不再是“死记”的对象，而是“推导”的结果——这正是换位子原理的威力。

9.5 评价：盲拧方法的演进与权衡

盲拧方法的演进 (OP $\rightarrow$ M2 $\rightarrow$ 3-Style) 清晰地展示了“作用范围”与“算法量”的权衡：

方法	作用类型	算法量	典型步数
Old Pochmann	对换 (2-cycle)	~20	200+
M2 (棱) + OP (角)	对换 (2-cycle)	~50	100–150
3-Style	三循环 (3-cycle)	~800	60–80

这一权衡与速拧方法 (LBL $\rightarrow$ CFOP $\rightarrow$ ZBLL) 的演进完全同构：用更大的算法量换取更少的步数与更高的效率。其背后的数学原理也相同——都是换位子原理的逐步深化应用。盲拧与速拧的差异仅在于“执行模式” (序列化 vs 即时识别)，而非“算法构造原理”。

10 计算机解法：Kociemba 两阶段算法

前述各章介绍的都是人类解法——它们在“记忆负担”与“步数效率”之间做权衡，以适应人类的识别与执行能力。本章介绍计算机解法——Kociemba 两阶段算法 [9, 10]，它不受人类记忆限制，可以找到接近最优 (≤ 22 步) 的解法。Kociemba 算法是 God's Number = 20 证明 (定理 2.3) 的核心工具，也是各类魔方求解软件 (如 Cube Explorer) 的算法基础。本章从第一性原理剖析 Kociemba 算法如何利用群论子群结构实现高效搜索。

10.1 思想：状态空间剪枝与子群分解

Kociemba 算法的核心思想是将求解问题分解为两个阶段，每个阶段在一个远小于原状态空间的子空间内搜索。这一分解的关键是利用魔方群的一个特殊子群 G_1 ，使得“进入 G_1 ”与“在 G_1 中求解”两个子问题都可在合理时间内完成。

Kociemba 两阶段的核心子群 G_1

定义子群

$$G_1 = \langle U, D, R_2, L_2, F_2, B_2 \rangle.$$

G_1 是 $\mathcal{R}$ 的一个子群，其元素满足：

1. 所有棱块已定向 (棱块朝向和为零且每个棱块朝向正确)；
2. 所有角块已定向 (角块朝向和为零且每个角块朝向正确)；
3. 4 个中层棱块 (FR, FL, BR, BL) 在中层 (位置可能需置换)。

$|G_1| \approx 1.95 \times 10^{10}$ ，远小于 $|\mathcal{R}| \approx 4.33 \times 10^{19}$ 。

G_1 的关键性质是：一旦进入 G_1 ，后续只需用 U, D, R_2, L_2, F_2, B_2 即可求解，且这些操作不会“离开” G_1 。这使得第二阶段的搜索空间被限制在 G_1 内，大幅缩小。而第一阶段的任务是将任意状态“送入” G_1 ，其搜索空间也通过启发式剪枝大幅缩小。

10.2 Phase 1：进入子群 G_1

Phase 1 的目标是从初始状态到达 G_1 中的某个状态。根据 G_1 的定义，这等价于：

1. 定向所有棱块 (消除“坏”棱块)；

2. 定向所有角块（使所有角块朝向正确）；
3. 将 4 个中层棱块送入中层（FR, FL, BR, BL 位置，不管顺序）。

Phase 1 使用**迭代加深 A* (IDA*) 搜索**，以一个**启发式函数**估计当前状态到 G_1 的距离。该启发式函数基于三个**剪枝表** (pruning tables)：

- **棱块定向剪枝表**：给定 12 个棱块的定向状态 ($2^{11} = 2048$ 种，因朝向和为偶数)，查表得到”还需多少步使所有棱块定向”的下界。
- **角块定向剪枝表**：给定 8 个角块的定向状态 ($3^7 = 2187$ 种)，查表得到”还需多少步使所有角块定向”的下界。
- **中层棱块位置剪枝表**：给定 4 个中层棱块的位置状态 ($\binom{12}{4} = 495$ 种)，查表得到”还需多少步将它们送入中层”的下界。

三个下界的最大值作为 Phase 1 的启发式值。由于三个剪枝表的笛卡尔积仅 $2048 \times 2187 \times 495 \approx 2.2 \times 10^9$ 种状态（可全部预计算并装入内存），Phase 1 的搜索极为高效，通常 5–12 步即可进入 G_1 。

10.3 Phase 2: 在 G_1 中求解

Phase 2 的目标是从 G_1 中的状态到达初始状态(还原),且只使用 G_1 的生成元 U, D, R_2 , L_2 , F_2 , B_2 。由于 G_1 的操作受限（只有 U, D 是 90° 转，其余都是 180° 转），Phase 2 的搜索空间远小于 $\mathcal{R}$ 。

Phase 2 同样使用 IDA* 搜索，基于两个剪枝表：

- **角块置换剪枝表**：给定 8 个角块的置换状态 ($8!/2 = 20160$ 种，因 G_1 中角块置换为偶)，查表得到”还需多少步归位角块”的下界。
- **棱块置换剪枝表**：给定 8 个非中层棱块的置换状态 ($8!/2 = 20160$ 种) 与 4 个中层棱块的置换状态 ($4!/2 = 12$ 种)，查表得到”还需多少步归位棱块”的下界。

Phase 2 通常 5–18 步即可完成求解。两阶段合计通常 12–22 步，远优于人类解法的 50–60 步。

10.4 启发式与剪枝表的设计原理

Kociemba 算法的效率核心在于**剪枝表的设计**。剪枝表的本质是**模式数据库(pattern database)**——预先计算所有子状态到目标的最短距离，存储为查找表。搜索时，通过查表得到”乐观下界”，从而剪去不可能改进的分支。

剪枝表的”子问题分解”原理

剪枝表之所以有效，是因为它将魔方状态分解为若干**独立的子问题**（棱块定向、角块定向、中层棱块位置等），每个子问题的状态空间远小于整体。对每个子问题预计算最短距离，再取最大值作为整体下界——这一”最大值”是合法的下界，因为任何操作对各子问题的影响是”同步”的（一步操作至少推进一个子问题，否则该步无效）。

Kociemba 算法的剪枝表总大小约几十 MB，可在现代计算机上全部装入内存，使得每次查表为 $O(1)$ 。这是 Kociemba 算法能在毫秒级求解的关键。

10.5 两阶段算法的优化与变体

基本的两阶段算法已能找到 ≤ 22 步的解法，但通过以下优化可进一步逼近最优：

1. **多解搜索**。不只找第一个解，而是找多个 Phase 1 + Phase 2 的组合，取最短。通过调整 Phase 1 的目标状态 (G_1 中有多个状态可达)，可以找到更短的整体解。

2. **两阶段边界优化**。允许 Phase 1 ”过冲”——即 Phase 1 不必严格停在 G_1 ，而是可以稍微超出，再在 Phase 2 中修正。这一”边界模糊化”可以找到更短的解。

3. **三阶段或更多阶段**。将求解分解为更多阶段（如 Thistlethwaite 的四阶段算法 [12]），每阶段进入更小的子群。阶段越多，每阶段搜索越快，但总步数可能增加。

10.6 评价：Kociemba 算法的意义与局限

意义：Kociemba 算法是魔方算法研究的里程碑。它不仅在实践中高效（毫秒级求解 ≤ 22 步），更在理论上为 God's Number = 20 的证明提供了核心工具——Rokicki 等人正是用 Kociemba 算法的变体验证了所有等价类代表状态都可在 20 步内还原。Kociemba 算法还深刻揭示了**子群结构**在魔力求解中的威力——通过将问题分解为”进入子群”与”在子群中求解”，可以将指数级搜索降为可行。

局限：Kociemba 算法的主要局限是**不保证最优**（找到的解可能比最优长 1-2 步）与**内存需求**（剪枝表需几十 MB）。对于追求严格最优的场合，需用更耗时的完整搜索（如 Korf 算法 [13]，使用所有 18 个生成元的完整模式数据库，但搜索时间可达数小时）。此外，Kociemba 算法对人类无直接指导意义——其解法路径往往”反直觉”，人类无法记忆与执行。但它的**子群分解思想**对人类方法有间接启发：ZZ 的”EO 优先”策略正是”进入 G_1 的棱块定向部分”的人类可执行版本。

11 方法关联与内在机制

前述各章分别介绍了 LBL、CFOP、Roux、ZZ、Petrus、盲拧系列与 Kociemba 算法。这些方法在策略层面千差万别——有的分层、有的块构建、有的定向优先、有的序列化执行——但在数学层面，它们都建立在第二章的群结构、不变量与第三章的换位子、共轭之上。本章从第一性原理视角，系统归纳各方法的共同数学骨架与策略权衡，揭示它们之间的内在关联。

11.1 共同的数学骨架

尽管各方法的外在形式不同，它们都遵循同一个”数学骨架”：

所有魔方方法的共同骨架

1. **子群递降**：所有方法都将求解过程组织为”从一个小子群逐步进入更小的子群”。LBL 从 $\mathcal{R}$ 进入”底层完成”子群，再进入”前两层完成”子群，再进入”顶层完成”子群 ($\{e\}$)。CFOP、Roux、ZZ、Petrus 同理，只是子群的选取不同。Kociemba 算法最显式地利用了这一点——Phase 1 进入 G_1 ，Phase 2 在 G_1 中求解。

2. **不变量保持**：每一步操作都保持三大不变量（定理 2.1）。这意味着每一步都是”合法操作”，不会产生非法状态。各方法的差异在于**如何**保持不变量——有的通过限制操作面（ZZ 只用保持定向的旋转），有的通过精心设计的算法（CFOP 的 OLL/PLL 算法天然保持不变量）。
3. **换位子构造**：所有”局部算法”（只影响少数小块的算法）本质上都是换位子或共轭的换位子。LBL 的 $RUR'U'$ 是换位子，CFOP 的 Sune 是角块朝向三循环换位子，PLL 的 T-Perm 是两个对换的复合（可分解为换位子），盲拧的 3-Style 更是直接以换位子为核心。换位子原理是所有方法算法构造的统一工具。

这一共同骨架的意义在于：**理解了群论与换位子原理，就理解了所有方法的”为什么”**。各方法的差异不在数学原理，而在策略选择——如何在子群递降的每一步选择操作、如何平衡步数与记忆。

11.2 分层 vs 块构建 vs 定向优先

各方法在策略层面可分为三大流派：

1. 分层流派 (LBL、CFOP)。核心策略是”从下到上逐层完成”。优点是概念清晰、步骤明确；缺点是”严格分层”限制了灵活性，且 F2L 阶段频繁的 R/U 转导致旋转量大。CFOP 通过”合并子步骤”（F2L、OLL、PLL）缓解了 LBL 的步数冗余，但”分层”的本质未变。

2. 块构建流派 (Roux、Petrus)。核心策略是”从小到大逐步构建块”。优点是步数效率高（块构建比分层更灵活，可利用更多”捷径”）、旋转少（Roux 几乎无旋转）；缺点是依赖直觉（块构建的规划比分层难）、TPS 上限低（直觉操作难以高速执行）。Roux 与 Petrus 的差异在于块的形状与位置——Roux 构建两个分离的 $1 \times 2 \times 3$ 块，Petrus 构建一个连续扩展的块。

3. 定向优先流派 (ZZ、Petrus 的 EO 部分)。核心策略是”尽早完成所有棱块定向，此后在’全定向’子群内操作”。优点是后续无定向干扰（求解更”干净”）、无旋转（ZZ）；缺点是 EO 规划难（识别坏棱块与规划翻转路径需大量练习）。ZZ 在最开始做 EO (EOLine)，Petrus 在 $2 \times 2 \times 3$ 后做 EO，时机不同但思想相同。

方法	流派	EO 时机	典型步数
LBL	分层	最后（顶十字）	100–150
CFOP	分层	最后（OLL）	55–60
Roux	块构建	LSE 阶段	45–55
ZZ	定向优先	最开始（EOLine）	55–60
Petrus	块构建 + 定向优先	$2 \times 2 \times 3$ 后	45–55

11.3 算法库 vs 即兴构造

各方法在”算法依赖度”上也有显著差异：

高算法依赖 (CFOP、3-Style)。这些方法依赖大量预记忆算法（CFOP 约 119 个，3-Style 约 800 个）。优点是执行快速（识别后直接执行，无需思考）；缺点是学习曲线陡（需数月记忆）、面对新状态时适应性差（若算法库未覆盖，需临时拆解）。

低算法依赖 (LBL、Roux 的 LSE、Petrus)。这些方法依赖直觉与模式识别，算法量少 (LBL 约 10 个，Roux 的 LSE 几乎无固定算法)。优点是学习门槛低、适应性强；缺点是执行速度受限于思考速度、TPS 上限低。

中等算法依赖 (ZZ、Roux 的 CMLL)。介于两者之间，部分阶段用算法 (ZZ 的 OLL/PLL、Roux 的 CMLL)，部分阶段用直觉 (ZZ 的 EOLine、Roux 的 LSE)。

算法与直觉的”光谱”

所有方法都位于”纯算法”与”纯直觉”的光谱上。CFOP 与 3-Style 靠近”纯算法”端，LBL 与 Roux-LSE 靠近”纯直觉”端。有趣的是，**顶级选手往往在”纯算法”方法中加入直觉元素**（如 CFOP 的”空槽 F2L”即兴决策），**在”纯直觉”方法中加入算法元素**（如 Roux 的 CMLL 算法库）。这一”光谱融合”是进阶的关键。

11.4 速度与记忆的权衡

各方法在”速度上限”与”记忆负担”之间存在根本权衡：

方法	记忆负担	速度上限
LBL	低 (~10 算法)	低 (~30 秒)
CFOP	高 (~119 算法)	极高 (sub-5 秒)
Roux	中 (~42 算法 + 直觉)	高 (sub-8 秒)
ZZ	中-高 (48-493 算法)	高 (sub-8 秒)
Petrus	低-中 (~30 算法 + 直觉)	中 (~15 秒)
3-Style 盲拧	极高 (~800 算法)	极高 (sub-20 秒盲拧)

这一权衡的数学根源在于**状态空间的庞大与人类工作记忆的有限**：魔方有 4.3×10^{19} 个状态，人类无法为每个状态记忆最优解。因此，方法必须在”覆盖更多状态”（高算法量）与”即兴处理新状态”（低算法量 + 直觉）之间选择。CFOP 选择前者（119 算法覆盖几乎所有 F2L/OLL/PLL 情况），Roux 选择后者（LSE 无算法，靠直觉处理）。

11.5 第一性原理的统一视角

从第一性原理视角，所有方法都可以视为在**换位子构造框架 (§3.5)**上的不同策略选择：

各方法在换位子框架上的策略选择	
方法	策略选择
LBL	严格分层，每层用简单换位子（如 R U R' U'）逐步构建
CFOP	合并分层，用更大的换位子（F2L、OLL、PLL）一次解决多块
Roux	块构建 + M 层换位子（LSE 用 M/U 组合即兴构造）
ZZ	EO 优先 + 全定向子群内的换位子（无 F/B 90° 转）
Petrus	块扩展 + EO + 保持定向的换位子
盲拧 3-Style	缓冲区 + 三循环换位子（最纯粹的换位子应用）
Kociemba	子群 G_1 分解 + 计算机搜索（不依赖人类可执行的换位子）

这一统一视角的深刻之处在于：**所有方法的差异都可以归结为”在换位子框架的哪一步做什么选择”**。LBL 选择”严格分层 + 简单换位子”，CFOP 选择”合并分层 + 大换位子”，Roux 选择”块构建 + M 层换位子”，ZZ 选择”EO 优先 + 保持定向换位子”，3-Style 选择”缓冲区 + 三循环换位子”。理解了这些选择背后的权衡，就能根据自身目标（速度？少步数？盲拧？理解？）选择最适合的方法，甚至创造新的方法。

从”学方法”到”理解方法”

本文的最终目标是帮助读者从”学习某个方法的步骤”升级到”理解所有方法的共同原理”。一旦达到这一层次，各方法不再是孤立的”公式集”，而是同一原理的不同应用。你将能够：

- (1) 根据目标选择最适合的方法；
- (2) 在不同方法间灵活切换或融合；
- (3) 面对陌生状态时从原理推导解法，而非依赖记忆；
- (4) 甚至创造新的方法或变体。这正是”第一性原理”视角的威力——它将”无限”的方法收敛为”有限”的原理。

12 策略总结与学习路径

本章基于前述各章的分析，给出面向不同目标的策略选择建议与学习路径。这些建议旨在帮助读者根据自身目标（入门、速拧、少步数、盲拧、理论研究）选择最适合的方法，并规划合理的学习进度。

12.1 学习路径建议

路径 1：入门到中级速拧（目标 sub-30 秒）。

- 1. 从 LBL 入门，掌握分层思想与基本算法（约 1-2 周）；
- 2. 过渡到 2-look CFOP（Cross + 2-look F2L + 2-look OLL + 2-look PLL），约 16 个算法（约 1-2 月）；

3. 逐步扩充到 full CFOP (F2L 41 + OLL 57 + PLL 21), 重点训练 F2L 的直觉识别与预判 (约 3-6 月);
4. 高级技巧: Cross 优化、F2L 空槽利用、OLL/PLL 识别加速 (持续训练)。

路径 2: 中级到高级速拧 (目标 sub-10 秒)。

1. 在 full CFOP 基础上, 深化 F2L 预判 (look-ahead) 与 Cross 盲做;
2. 考虑学习 COLL/CLL 以减少 LL 步数;
3. 若对单手速拧感兴趣, 可转向 Roux (M 转优势) 或 ZZ (无旋转优势);
4. 持续训练 TPS (turns per second) 与执行流畅度。

路径 3: 少步数赛 (Fewest Moves, 目标 sub-30 步)。

1. 学习 Petrus 或 Roux 的块构建策略 (步数效率高);
2. 掌握 EO 技术 (ZZ 的 EOLine 或 Petrus 的 EO), 减少定向干扰;
3. 学习 NISS (Normal-Inverse Scramble Switch) 等少步数赛专用技巧;
4. 研究计算机解法 (Kociemba) 的思路, 启发人类可执行的优化。

路径 4: 盲拧 (目标 sub-60 秒)。

1. 从 Old Pochmann 入门, 掌握“缓冲区-循环”模型与 Y-Perm/T-Perm (约 1 月);
2. 棱块升级到 M2, 提升棱块效率 (约 1-2 月);
3. 逐步学习 3-Style, 从高频目标对开始, 逐步扩充算法库 (约 6-12 月);
4. 训练记忆编码与执行流畅度。

路径 5: 理论研究。

1. 深入学习群论 (推荐 Joyner[3]);
2. 研究换位子与共轭的构造技巧 (推荐 Frey-Singmaster[2]);
3. 了解计算机求解算法 (Kociemba[9]、Korf[13]);
4. 探索高阶魔方 (4×4 、 5×5) 与异形魔方的群论结构。

12.2 不同目标的策略选择

目标-方法匹配表	
目标	推荐方法
入门理解	LBL（概念清晰，门槛低）
双手速拧	CFOP（生态成熟，速度上限高）
单手速拧	Roux 或 ZZ（旋转少，人体工学好）
少步数赛	Petrus 或 Roux + EO 技术（步数效率高）
盲拧入门	Old Pochmann（算法少，可靠性高）
盲拧竞速	3-Style（步数少，速度极快）
理论研究	群论 + 换位子 + Kociemba（理解原理）

12.3 进阶方向

- 掌握上述方法后，可探索以下进阶方向：
- 1. **方法融合**。不同方法的技巧可以融合。例如，CFOP 的 F2L 可以借鉴 Roux 的块构建思想（”成对构建”而非”先角后棱”）；Roux 的 LSE 可以借鉴 ZZ 的 EO 技术（提前定向以简化 LSE）。
 - 2. **高阶魔方**。4×4、5×5 等高阶魔方引入了”中心块可移动”与”棱块配对”的新问题，其群论结构更复杂。但核心原理（子群递降、换位子构造）仍然适用。
 - 3. **异形魔方**。Megaminx、Pyraminx、Square-1 等异形魔方有不同的几何结构与群结构，但换位子与共轭仍是核心工具。
 - 4. **算法生成与优化**。利用计算机自动生成与优化算法（如找到更短的 OLL/PLL 算法），是魔方研究的前沿方向。
 - 5. **数学深化**。魔方群的精细结构（如共轭类、中心、换位子子群）有丰富的数学内容，值得深入研究。

12.4 结语：第一性原理的力量

本文从第一性原理出发，系统梳理了魔方的数学结构与主流解法。我们看到，尽管魔方有 4.3×10^{19} 个状态、数十种解法方法，但其核心原理极为简洁——**群结构、不变量、换位子与共轭**。这三个原理构成了所有方法的公共基底，各方法的差异仅在于”如何在这三个原理之上做策略选择”。

这一”从无限到有限”的收敛，正是第一性原理思维的力量。它不仅适用于魔方，也适用于任何复杂系统的理解——**找到不可约的基本原理，然后看所有复杂性如何从这些原理中涌现**。希望本文不仅帮助读者更好地理解与求解魔方，更能启发读者在其他领域运用第一性原理思维，从纷繁的现象中洞察简洁的本质。

参考文献

[1] D. Singmaster, *Notes on Rubik's Magic Cube*, Enslow Publishers, 1980.

- [2] A. H. Frey Jr. and D. Singmaster, *Handbook of Cubik Math*, Enslow Publishers, 1982.
- [3] D. Joyner, *Adventures in Group Theory: Rubik's Cube, Merlin's Machine, and Other Mathematical Toys*, 2nd ed., Johns Hopkins University Press, 2008.
- [4] J. Fridrich, "My system for solving Rubik's Cube", *Speedsolving Rubik's Cube* (online resource), 1997. <https://www.ws.binghamton.edu/fridrich/cube.html>
- [5] G. Roux, "Roux Method", *Speedsolving Wiki*, 2003. https://www.speedsolving.com/wiki/index.php/Roux_method
- [6] Z. Zborowski and R. Bruchem, "ZZ Method", *Speedsolving Wiki*, 2006. https://www.speedsolving.com/wiki/index.php/ZZ_method
- [7] L. Petrus, "Petrus Method", *Lars Petrus' Rubik's Cube Page*, 1981. <https://lar5.com/cube/>
- [8] S. Pochmann, "Blindfold Solving", *Stefan Pochmann's Rubik's Cube Page*, 2004. <https://www.stefan-pochmann.info/spocc/blindfold/>
- [9] H. Kociemba, "The Two-Phase Algorithm", *Cube Explorer* (online resource), 1992. <https://kociemba.org/cube.htm>
- [10] H. Kociemba, "Solving the Rubik's Cube Optimally", in *The Mathematics of Various Entertaining Subjects*, Vol. 2, Princeton University Press, 2017, pp. 133–152.
- [11] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, "The Diameter of the Rubik's Cube Group Is Twenty", *SIAM Journal on Discrete Mathematics*, 27(2): 1082–1105, 2014.
- [12] J. Thistlethwaite, "The 45-move strategy", unpublished manuscript, 1981.
- [13] R. E. Korf, "Finding Optimal Solutions to Rubik's Cube Using Pattern Databases", *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*, 1997, pp. 700–705.
- [14] D. Harris, *Speedsolving the Cube: Easy-to-Follow, Step-by-Step Instructions for Many Popular 3-D Puzzles*, Sterling, 2008.
- [15] E. Rubik, T. Varga, G. Keri, G. Marx, and T. Vekerdy, *Rubik's Cubic Compendium*, Oxford University Press, 1987.